



UNIVERSIDADE FEDERAL DO CEARÁ

CAMPUS QUIXADÁ

Trabalho 2

Sistema de Gerenciamento de Estoque de Suplementos

1. Introdução

Este relatório descreve o serviço remoto implementado no Trabalho 2 da disciplina de Sistemas Distribuídos. O objetivo foi reimplementar o sistema de controle de estoque de suplementos — desenvolvido no Trabalho 1 — utilizando o modelo de comunicação cliente-servidor baseado no protocolo de requisição-resposta.

A comunicação entre cliente e servidor foi implementada manualmente, sem o uso de sockets diretamente expostos ao cliente, e toda passagem de dados utiliza representação externa no formato JSON.

2. Descrição do Serviço Remoto

O serviço remoto expõe operações de gerenciamento de um estoque de suplementos alimentares. O servidor mantém o estado do estoque em memória e atende requisições de múltiplos clientes. O cliente interage por meio de um menu textual e não possui lógica de negócio, toda operação é delegada ao servidor via invocação remota.

Métodos Disponíveis para Invocação Remota

A interface `EstoqueService` define os 7 métodos expostos remotamente:

#	Método	Parâmetros	Retorno	Passagem
1	<code>adicionarSuplemento</code>	<code>Suplemento s</code>	<code>String</code> (confirmação)	Por valor (JSON)
2	<code>adicionarLote</code>	<code>String nome,</code> <code>Lote lote</code>	<code>String</code> (confirmação)	Por valor (JSON)
3	<code>buscarSuplemento</code>	<code>String nome</code>	<code>Suplemento</code>	Por referência (<code>RemoteObjectRef</code>)
4	<code>listarEstoque</code>	—	<code>List<Suplemento></code>	Por valor (JSON)
5	<code>listarVencidos</code>	—	<code>List<Suplemento></code>	Por valor (JSON)
6	<code>listarProximosAoVencer</code>	<code>int dias</code>	<code>List<Suplemento></code>	Por valor (JSON)
7	<code>removerSuplemento</code>	<code>String nome</code>	<code>String</code> (confirmação)	Por valor (JSON)

3. Protocolo de Requisição-Resposta

3.1 Estrutura das Mensagens

As mensagens seguem o formato:

Campo	<code>RequestMessage</code>	<code>ReplyMessage</code>
<code>messageType</code>	0 (Request)	1 (Reply)

Campo	RequestMessage	ReplyMessage
requestId	Inteiro auto-incrementado	Mesmo ID da requisição correspondente
objectReference	Nome do objeto remoto (ex: "EstoqueService")	—
methodId	Nome do método (ex: "adicionarSuplemento")	—
arguments / result	Argumentos serializados em JSON (byte[])	Resultado serializado em JSON (byte[])
status	—	"OK" ou "ERROR"

3.2 Fluxo de uma Chamada Remota

O fluxo completo de uma invocação remota:

1. O cliente chama um método no EstoqueProxy (ex: buscarSuplemento("Whey Gold"))
2. O proxy serializa os argumentos em JSON via JsonSerializer e cria um RequestMessage
3. CommunicationModule.doOperation() abre uma conexão TCP com o servidor e envia o RequestMessage serializado
4. No servidor, RequestReplyProtocol.getRequest() recebe e desserializa a mensagem
5. O Dispatcher identifica o methodId e desempacota os argumentos JSON
6. O método correspondente é invocado no EstoqueSkeletonImpl, que executa sobre o Estoque
7. O resultado é serializado em JSON e empacotado num ReplyMessage
8. RequestReplyProtocol.sendReply() envia a resposta de volta ao cliente
9. CommunicationModule recebe o ReplyMessage e retorna os bytes do resultado
10. O proxy desserializa o JSON e devolve o objeto ao código do cliente

4. Passagem de Parâmetros

4.1 Passagem por Valor (Representação Externa de Dados)

Toda troca de dados entre cliente e servidor é feita por valor, utilizando JSON como representação externa de dados. A classe JsonSerializer implementa a serialização e desserialização manualmente, sem dependências externas.

Exemplo de serialização de um suplemento:

```
{
  "tipo": "Whey Protein",
  "nome": "Whey Gold Standard",
  "marca": "Optimum Nutrition",
  "preco": 299.9,
  "proteinas": 25.0,
  "sabor": "Chocolate",
}
```

```
"lotes": [{"codigo": "L001", "quantidade": 50, ...}]
}
```

4.2 Passagem por Referência

A classe `RemoteObjectRef` representa a referência a um objeto remoto, encapsulando `host`, `porta` e `nome` do objeto. O `EstoqueProxy` mantém uma instância de `RemoteObjectRef` que é passada em cada chamada a `doOperation()`, identificando o objeto remoto no servidor sem transferir seu estado.

O método `buscarSuplemento` ilustra a passagem por referência: o cliente solicita um objeto pelo nome; o servidor localiza o objeto real em memória e retorna seus dados — o objeto físico permanece no servidor.

5. Considerações Finais

O sistema implementado respeita integralmente o protocolo requisição-resposta, com os três métodos centrais (`doOperation`, `getRequest`, `sendReply`) e o empacotamento das mensagens conforme especificado. A serialização JSON manual garante a representação externa de dados sem dependências externas, tornando o projeto autocontido.